Kubernetes — 100 Scenarios, Explained

EngiDock · Learn once, own for life.

Real situations a Kubernetes engineer faces, each with the concept behind it, the exact manifest or kubectl command, the output you should see, and a clear line-by-line explanation. Built to teach the *why*, not just the *how*.

100 scenarios · **11 categories** · Concept + explanation on every entry · Beginner → Expert

---

Contents

1. Core Concepts & kubectl — 9
2. Pods & Containers — 10
3. Deployments & ReplicaSets — 10
4. Services & Networking — 10
5. ConfigMaps & Secrets — 9
6. Storage: Volumes, PV & PVC — 9
7. Scheduling & Resources — 10
8. Health, Probes & Autoscaling — 9
9. RBAC & Security — 8
10. Observability & Troubleshooting — 8
11. Advanced: Workloads & Tooling — 8

Core Concepts & kubectl

001. Confirm the cluster and client are healthy

Scenario — You've just pointed kubectl at a cluster and want to confirm it works before doing anything.

Concept — kubectl is a client that talks to the cluster's API server, the single front door to Kubernetes. The client and server versions can differ; a small skew is fine, a large one can cause odd behaviour.

Command / Code

```
kubectl version --short
kubectl cluster-info
```

Output

```
Client Version: v1.30.2
Server Version: v1.30.1
Kubernetes control plane is running at https://10.0.0.1:6443
CoreDNS is running at https://10.0.0.1:6443/...
```

Explanation

- **version** shows both the kubectl client and the API server versions.
- **cluster-info** confirms the control plane endpoint is reachable.
- If the server line errors, your kubeconfig or network to the API server is wrong.

> **Key idea:** Everything kubectl does goes through the API server — it's the cluster's single source of truth.

---

002. List the nodes that run your workloads

Scenario — You need to see the worker machines and whether they're ready to schedule pods.

Concept — Nodes are the machines (VMs or bare metal) that run pods. The scheduler only places pods on nodes in the Ready state, so node health directly affects your workloads.

Command / Code

```
kubectl get nodes -o wide
```

Output

```
NAME       STATUS   ROLES           AGE   VERSION   INTERNAL-IP
cp-1       Ready    control-plane   40d   v1.30.1   10.0.0.1
worker-1   Ready    <none>          40d   v1.30.1   10.0.0.11
worker-2   NotReady <none>          40d   v1.30.1   10.0.0.12
```

Explanation

- **get nodes** lists cluster machines; **-o wide** adds IPs and more.
- **Ready** means the node can accept pods; **NotReady** means it can't.
- The scheduler avoids NotReady nodes, so pods may pile up as Pending.

> **Key idea:** A NotReady node removes capacity — pods there get rescheduled elsewhere if possible.

---

003. Organise resources with namespaces

Scenario — You want to isolate the team's app from others sharing the same cluster.

Concept — Namespaces partition a cluster into virtual sub-clusters for isolation, quotas, and access control. Most resources live in a namespace; a few (like nodes) are cluster-wide.

Command / Code

```
kubectl create namespace nimbus
kubectl get pods -n nimbus
kubectl config set-context --current --namespace nimbus
```

Output

```
namespace/nimbus created
No resources found in nimbus namespace.
# default namespace for this context is now nimbus
```

Explanation

- **create namespace** makes an isolated space for your resources.
- **-n nimbus** scopes a command to that namespace.
- Setting the context's namespace saves typing -n on every command.

> **Key idea:** Set your context's namespace once so you stop repeating -n everywhere.

---

004. Inspect any resource with get and describe

Scenario — A pod is misbehaving and you need both a quick status and full detail.

Concept — get gives a compact list view; describe shows full detail including events, which are often the key to diagnosing problems. Events explain *why* something is in its current state.

Command / Code

```
kubectl get pod web-1 -o wide
kubectl describe pod web-1 | tail -6
```

Output

```
NAME    READY   STATUS    RESTARTS   AGE   IP           NODE
web-1   1/1     Running   0          5m    10.244.1.7   worker-1
Events:
  Normal  Scheduled  5m   default-scheduler  Successfully assigned nimbus/web-1
  Normal  Pulled     5m   kubelet            Container image already present
```

Explanation

- **get** is the fast status view; **-o wide** adds IP and node.
- **describe** prints full spec plus an **Events** section.
- Events are usually where the real reason for a problem appears.

> **Key idea:** When something's wrong, read the Events at the bottom of describe first.

---

005. Prefer declarative apply over imperative run

Scenario — You want your resources reproducible and version-controlled, not created by ad-hoc commands.

Concept — Imperative commands (run, create) act once; declarative apply reconciles the cluster to match a YAML file you can commit. Declarative is the GitOps-friendly, reproducible way.

Command / Code

```
kubectl run tmp --image=nginx --restart=Never   # imperative, one-off
kubectl apply -f deploy.yaml                    # declarative, reproducible
```

Output

```
pod/tmp created
deployment.apps/web created
# apply can be re-run safely; it reconciles to the file
```

Explanation

- **run** creates a single resource imperatively — fine for quick tests.
- **apply -f** makes the cluster match a declarative manifest.
- apply is idempotent and diffable — the basis of GitOps.

> **Key idea:** Use imperative commands to explore; use apply -f for anything real.

---

006. Discover fields with kubectl explain

Scenario — You can't remember the exact path for a field in a manifest.

Concept — kubectl explain documents the schema of any resource straight from the API server, so you can look up valid fields and types without leaving the terminal.

Command / Code

```
kubectl explain deployment.spec.strategy
kubectl explain pod.spec.containers.resources --recursive | head
```

Output

```
KIND: Deployment
FIELD: strategy <Object>
DESCRIPTION:
  The deployment strategy to use to replace existing pods...
FIELDS:
  rollingUpdate  <Object>
  type           <string>
```

Explanation

- **explain** prints the schema and docs for a field path.
- **--recursive** expands nested fields for the full structure.
- It's authoritative because it comes from the live API server.

> **Key idea:** kubectl explain is built-in documentation for every manifest field.

---

007. Switch between clusters with contexts

Scenario — You manage staging and prod clusters and must be sure which one you're targeting.

Concept — A kubeconfig context bundles a cluster, user, and namespace. Switching contexts changes which cluster kubectl talks to — checking it prevents dangerous mistakes.

Command / Code

```
kubectl config get-contexts
kubectl config use-context prod
```

Output

```
CURRENT   NAME      CLUSTER   NAMESPACE
*         staging   staging   nimbus
          prod      prod      nimbus
Switched to context "prod".
```

Explanation

- **get-contexts** lists available contexts; the ***** marks the current one.
- **use-context** switches which cluster commands hit.
- Always confirm your context before running destructive commands.

> **Key idea:** Confirm your context before anything destructive — staging vs prod mistakes are costly.

---

008. Select resources with labels

Scenario — You want to act on just the frontend pods among many in a namespace.

Concept — Labels are key/value tags on objects; selectors query by them. This label-and-select model is how Services, Deployments, and you target sets of objects.

Command / Code

```
kubectl get pods -l app=web,tier=frontend
kubectl label pod web-1 canary=true
```

Output

```
NAME    READY   STATUS    AGE
web-1   1/1     Running   6m
web-2   1/1     Running   6m
pod/web-1 labeled
```

Explanation

- **-l app=web,tier=frontend** selects pods matching both labels.
- **kubectl label** adds or changes a label on an object.

- Labels are the glue Kubernetes uses to connect Services to pods.

> **Key idea:** Labels + selectors are the core mechanism linking Services, Deployments, and pods.

---

009. Generate manifests with dry-run

Scenario — You want a correct starting YAML without writing it from scratch.

Concept — --dry-run=client with -o yaml builds a valid manifest from an imperative command without touching the cluster. It's the fastest way to scaffold correct YAML.

Command / Code

```
kubectl create deployment web --image=nginx:1.27 \
  --dry-run=client -o yaml > deploy.yaml
head -8 deploy.yaml
```

Output

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 1
  selector:
    matchLabels: { app: web }
```

Explanation

- **--dry-run=client** builds the object locally, changing nothing.
- **-o yaml** prints the manifest you can save and edit.
- This scaffolds correct structure so you don't hand-write boilerplate.

> **Key idea:** Scaffold any manifest with create ... --dry-run=client -o yaml.

Pods & Containers

010. Run your first pod from a manifest

Scenario — You want to run a single container in the cluster declaratively.

Concept — A pod is the smallest deployable unit — one or more containers that share a network and storage context. You rarely create bare pods in production, but understanding them is foundational.

`pod.yaml`

```yaml
apiVersion: v1
kind: Pod
metadata: { name: web, labels: { app: web } }
spec:
  containers:
    - name: nginx
      image: nginx:1.27-alpine
      ports: [ { containerPort: 80 } ]
```

Output

```
kubectl apply -f pod.yaml
pod/web created
kubectl get pod web
web   1/1   Running   0   8s
```

Explanation

- **kind: Pod** with one container is the simplest workload.
- Containers in a pod share the pod's IP and can talk over localhost.
- **1/1** means one of one container is ready and running.

> **Key idea:** Pods are the atomic unit — higher objects (Deployments) manage pods for you.

---

011. Add a sidecar container to a pod

Scenario — Your app writes logs to a file and you want a sidecar to ship them.

Concept — A pod can hold multiple containers that share volumes and the network. The sidecar pattern runs a helper container (logging, proxy) alongside the main app.

`pod-sidecar.yaml`

```yaml
spec:
  containers:
    - name: app
      image: nimbus-app:1.0
      volumeMounts: [ { name: logs, mountPath: /var/log } ]
    - name: log-shipper
      image: fluent-bit:2.2
      volumeMounts: [ { name: logs, mountPath: /var/log } ]
  volumes: [ { name: logs, emptyDir: {} } ]
```

Output

```
pod/app created
kubectl get pod app
app   2/2   Running   0   12s
```

Explanation

- Both containers mount the same **logs** volume to share files.
- The app writes logs; the sidecar reads and ships them.
- **2/2** confirms both containers are ready.

> **Key idea:** Sidecars share the pod's volumes and network — ideal for logging or proxies.

---

012. Prepare state with init containers

Scenario — Your app must wait for a database and run a migration before it starts.

Concept — Init containers run to completion, in order, before app containers start. They handle setup (waiting on dependencies, migrations) that must finish first.

`pod-init.yaml`

```
spec:
  initContainers:
    - name: wait-db
      image: busybox
      command: ['sh','-c','until nc -z db 5432; do sleep 2; done']
  containers:
    - name: app
      image: nimbus-app:1.0
```

Output

```
kubectl get pod app
app   0/1   Init:0/1   0   4s
app   1/1   Running   0   20s
```

Explanation

- **initContainers** run and must succeed before app containers start.
- Here it blocks until the database port is reachable.
- The **Init:0/1** status shows init containers still running.

> **Key idea:** Init containers guarantee prerequisites are met before your app starts.

---

013. Exec into a running container

Scenario — You need a shell inside a pod to inspect files or test connectivity.

Concept — kubectl exec runs a command inside a container, optionally interactively. It's the primary way to debug from within a pod, similar to docker exec.

Command / Code

```
kubectl exec -it web -- sh
# inside:
nginx -t
exit
```

Output

```
/ # nginx -t
nginx: configuration file /etc/nginx/nginx.conf test is successful
/ # exit
```

Explanation

- **-it** gives an interactive TTY; **--** separates the command to run.
- For multi-container pods add **-c** <name> to pick a container.
- exec starts a process inside the existing container — it doesn't restart it.

> **Key idea:** Add -c <name> when execing into a multi-container pod.

---

014. Read logs, including previous crashes

Scenario — A container keeps restarting and you need the logs from the crashed instance.

Concept — kubectl logs streams a container's stdout/stderr. The --previous flag shows the last terminated container's logs — essential for diagnosing crash loops.

Command / Code

```
kubectl logs web -c nginx --tail 3
kubectl logs web --previous
```

Output

```
10.244.1.1 - GET / 200
10.244.1.1 - GET /health 200
# --previous shows logs from the crashed container
panic: cannot connect to db
```

Explanation

- **--tail 3** limits output; **-c** picks a container in a multi-container pod.
- **--previous** reveals why the last instance died — key for CrashLoopBackOff.
- Apps should log to stdout/stderr so kubectl logs captures them.

> **Key idea:** logs --previous is how you see why a crash-looping container died.

---

015. Inject configuration via environment variables

Scenario — Your container needs a log level and an environment name at runtime.

Concept — env sets environment variables directly in the container spec. It's the simplest config mechanism; for shared or sensitive values, prefer ConfigMaps and Secrets.

`pod-env.yaml`

```
spec:
  containers:
    - name: app
      image: nimbus-app:1.0
      env:
        - { name: LOG_LEVEL, value: debug }
        - { name: APP_ENV, value: staging }
```

Output

```
kubectl exec app -- printenv LOG_LEVEL APP_ENV
debug
staging
```

Explanation

- **env** defines variables the container sees at startup.
- Inline values are fine for simple, non-secret config.
- For reuse across pods, source them from a ConfigMap instead.

> **Key idea:** Inline env is fine for simple config; use ConfigMaps/Secrets for shared or secret data.

---

016. Set resource requests and limits

Scenario — One pod is starving neighbours of CPU and memory on a node.

Concept — Requests are what the scheduler reserves; limits are the hard cap the kubelet enforces. Requests drive placement; exceeding a memory limit triggers an OOM kill.

`pod-resources.yaml`

```
    resources:
      requests: { cpu: 250m, memory: 256Mi }
      limits:   { cpu: '1',  memory: 512Mi }
```

Output

```
kubectl describe pod app | grep -A4 Limits
    Limits:  cpu: 1, memory: 512Mi
    Requests: cpu: 250m, memory: 256Mi
```

Explanation

- **requests** reserve capacity and influence which node is chosen.
- **limits** cap usage; exceeding the memory limit causes an OOMKill.
- CPU over the limit is throttled, not killed.

> **Key idea:** Requests affect scheduling; limits protect the node — always set both in production.

---

017. Override the container command and args

Scenario — You need a container to run a one-off command instead of its default entrypoint.

Concept — command overrides the image's ENTRYPOINT and args overrides its CMD. This lets you repurpose an image without rebuilding it.

`pod-cmd.yaml`

```
spec:
  restartPolicy: Never
  containers:
    - name: job
      image: busybox
      command: ['sh','-c']
      args: ['echo processing; sleep 2; echo done']
```

Output

```
kubectl logs job
processing
done
```

Explanation

- **command** replaces the image ENTRYPOINT; **args** replaces CMD.
- **restartPolicy: Never** suits a run-once task.
- This reuses a general image for a specific task.

> **Key idea:** command overrides ENTRYPOINT, args overrides CMD — no image rebuild needed.

---

018. Debug a distroless pod with an ephemeral container

Scenario — A pod has no shell (distroless) and you must debug it live.

Concept — kubectl debug attaches an ephemeral container to a running pod, sharing its namespaces. It gives you tools inside a pod that has none, without restarting it.

Command / Code

```
kubectl debug -it web --image=busybox --target=nginx -- sh
# inside:
wget -qO- localhost:80 | head -1
```

Output

```
Defaulting debug container name to debugger-8x2...
/ # wget -qO- localhost:80 | head -1
<!DOCTYPE html>
```

Explanation

- **kubectl debug** injects a temporary container into the pod.
- **--target** shares the target container's process namespace.
- The pod keeps running — the ephemeral container is just for debugging.

> **Key idea:** kubectl debug adds tools to a shell-less pod without restarting it.

---

019. Understand pod restart policy

Scenario — A short-lived task pod keeps restarting when you only wanted it to run once.

Concept — restartPolicy (Always, OnFailure, Never) controls whether the kubelet restarts containers. Deployments require Always; Jobs typically use OnFailure or Never.

`pod-restart.yaml`

```
spec:
  restartPolicy: OnFailure
  containers:
    - name: task
      image: busybox
      command: ['sh','-c','exit 1']
```

Output

```
kubectl get pod task
task   0/1   CrashLoopBackOff   3   40s
# OnFailure keeps retrying a failing command
```

Explanation

- **OnFailure** restarts the container only on non-zero exit.
- A command that always fails yields **CrashLoopBackOff** with backoff delays.
- Use **Never** for a true one-shot that shouldn't retry.

> **Key idea:** CrashLoopBackOff means it keeps failing and restarting — check logs --previous.

Deployments & ReplicaSets

020. Create a Deployment

Scenario — You want a self-healing, scalable set of identical app pods.

Concept — A Deployment manages a ReplicaSet, which keeps a desired number of pod replicas running. You declare the desired state; the controller continuously reconciles reality to match.

`deploy.yaml`

```
apiVersion: apps/v1
kind: Deployment
metadata: { name: web }
spec:
  replicas: 3
  selector: { matchLabels: { app: web } }
  template:
    metadata: { labels: { app: web } }
    spec:
      containers: [ { name: nginx, image: nginx:1.27-alpine } ]
```

Output

```
deployment.apps/web created
kubectl get deploy web
NAME   READY   UP-TO-DATE   AVAILABLE
web    3/3     3            3
```

Explanation

- **replicas: 3** declares the desired pod count.
- **selector** must match the pod template's **labels**.
- The Deployment creates a ReplicaSet that maintains the three pods.

> **Key idea:** You declare desired state; the controller makes reality match, self-healing included.

---

021. Scale a Deployment up or down

Scenario — Traffic is rising and you need more replicas quickly.

Concept — Scaling changes the desired replica count; the ReplicaSet adds or removes pods to match. It's instant and safe for stateless workloads.

Command / Code

```
kubectl scale deployment web --replicas=5
kubectl get pods -l app=web --no-headers | wc -l
```

Output

```
deployment.apps/web scaled
5
# two new pods created to reach the desired 5
```

Explanation

- **scale --replicas=5** updates the desired count.
- The ReplicaSet creates the extra pods automatically.
- For declarative workflows, edit replicas in the YAML and apply instead.

> **Key idea:** Scaling is instant for stateless apps; stateful ones need more care.

---

022. Roll out a new image version

Scenario — You need to deploy a new app version with zero downtime.

Concept — Updating a Deployment's image triggers a rolling update: new pods come up and old ones are removed gradually, keeping the app available throughout.

Command / Code

```
kubectl set image deployment/web nginx=nginx:1.27.1-alpine
kubectl rollout status deployment/web
```

Output

```
deployment.apps/web image updated
Waiting for rollout: 2 of 3 updated...
deployment "web" successfully rolled out
```

Explanation

- **set image** changes the container image, starting a rollout.
- **rollout status** blocks until the update completes.
- New pods must become Ready before old ones are removed.

> **Key idea:** Rolling updates keep the app serving traffic while replacing pods gradually.

---

023. Roll back a bad deployment

Scenario — A new version is failing and you must revert immediately.

Concept — Kubernetes keeps a rollout history of ReplicaSets. rollout undo reverts to the previous (or a specific) revision, making rollback fast and safe.

Command / Code

```
kubectl rollout history deployment/web
kubectl rollout undo deployment/web
```

Output

```
REVISION  CHANGE-CAUSE
1         <none>
2         <none>
deployment.apps/web rolled back
```

Explanation

- **rollout history** lists past revisions of the Deployment.
- **rollout undo** reverts to the previous ReplicaSet.
- Add **--to-revision=N** to jump to a specific earlier version.

> **Key idea:** rollout undo is your instant safety net for a bad release.

---

024. See the ReplicaSet behind a Deployment

Scenario — You want to understand what actually creates and owns your pods.

Concept — A Deployment doesn't create pods directly; it creates a ReplicaSet per revision, and the ReplicaSet creates the pods. Each rollout produces a new ReplicaSet.

Command / Code

```
kubectl get rs -l app=web
```

Output

```
NAME             DESIRED   CURRENT   READY   AGE
web-6d4c8b9f7    3         3         3       2m
web-5f9a1c2e8    0         0         0       9m
```

Explanation

- Each Deployment revision creates a new **ReplicaSet**.
- The current RS has the desired replicas; old ones scale to 0.
- Old ReplicaSets are kept for rollback history.

> **Key idea:** Deployment → ReplicaSet → Pods. Rollbacks just re-scale an old ReplicaSet.

---

025. Tune the rolling update strategy

Scenario — You want tighter control over how many pods change at once during updates.

Concept — maxUnavailable and maxSurge control a RollingUpdate's pace: how many pods can be down and how many extra can be created. They balance update speed against availability.

`deploy-strategy.yaml`

```yaml
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1
      maxSurge: 1
```

Output

```
# during rollout: at most 1 pod down, at most 1 extra pod
kubectl rollout status deployment/web
deployment "web" successfully rolled out
```

Explanation

- **maxUnavailable: 1** keeps all but one replica serving during the update.
- **maxSurge: 1** allows one extra pod above desired while rolling.
- Lower values are safer but slower; higher values are faster but riskier.

> **Key idea:** maxUnavailable + maxSurge trade update speed against availability.

---

026. Use the Recreate strategy when needed

Scenario — Your app can't run two versions at once (e.g. a schema-locking migration).

Concept — The Recreate strategy terminates all old pods before creating new ones. It causes downtime but guarantees no overlap between versions.

`deploy-recreate.yaml`

```yaml
spec:
  strategy: { type: Recreate }
  replicas: 3
  template:
    spec:
      containers: [ { name: app, image: nimbus-app:2.0 } ]
```

Output

```
# all old pods terminated first...
# ...then new pods created (brief downtime)
deployment "app" successfully rolled out
```

Explanation

- **Recreate** stops every old pod before starting new ones.
- This avoids running two versions simultaneously.
- Accept the brief downtime when version overlap is unsafe.

> **Key idea:** Recreate = no version overlap, at the cost of a short outage.

027. Pause and resume a rollout

Scenario — You want to make several changes but roll them out as one, not one-by-one.

Concept — Pausing a Deployment stops it from rolling out changes. You can batch multiple edits, then resume to apply them all in a single rollout.

Command / Code

```
kubectl rollout pause deployment/web
kubectl set image deployment/web nginx=nginx:1.27.2-alpine
kubectl set resources deployment/web -c nginx --limits=memory=256Mi
kubectl rollout resume deployment/web
```

Output

```
deployment.apps/web paused
deployment.apps/web image updated
deployment.apps/web resource requirements updated
deployment.apps/web resumed
```

Explanation

- **pause** freezes rollouts so edits don't each trigger one.
- You batch image and resource changes while paused.
- **resume** applies everything in a single rollout.

> **Key idea:** Pause, batch multiple changes, then resume for one clean rollout.

---

028. Match pods with the right selector

Scenario — Your Deployment reports pods but a Service isn't finding them.

Concept — A Deployment's selector.matchLabels must match the pod template labels, and a Service selects the same labels. A mismatch silently breaks the link between them.

Command / Code

```
# Deployment
selector: { matchLabels: { app: web } }
template: { metadata: { labels: { app: web, tier: frontend } } }
# Service must select a subset that pods actually have
# selector: { app: web }   (matches)
```

Output

```
# Service endpoints populate because app=web matches
# if Service selected app=api, endpoints would be empty
```

Explanation

- The Deployment **selector** must be a subset of the pod template labels.
- A Service uses its own selector to find those same pods.
- Label typos are a top cause of 'Service has no endpoints'.

> **Key idea:** Selector/label mismatches silently break Service-to-pod wiring — check labels first.

---

029. Apply changes declaratively

Scenario — You edited the Deployment YAML and want the cluster to match it.

Concept — apply reconciles the live object to your manifest, computing the diff for you. It's idempotent and the recommended way to manage resources over time.

Command / Code

```
kubectl apply -f deploy.yaml
kubectl diff -f deploy.yaml || true
```

Output

```
deployment.apps/web configured
# kubectl diff shows nothing after apply — cluster matches file
```

Explanation

- **apply** updates only what changed and records the manifest.
- **kubectl diff** previews changes before applying — like a dry run.
- Re-applying an unchanged file is a no-op (idempotent).

> **Key idea:** kubectl diff -f before apply -f previews exactly what will change.

Services & Networking

030. Expose pods with a ClusterIP Service

Scenario — Your frontend needs a stable address to reach backend pods whose IPs keep changing.

Concept — Pods are ephemeral and their IPs change. A Service gives a stable virtual IP and DNS name that load-balances across the pods matching its selector.

`svc.yaml`

```yaml
apiVersion: v1
kind: Service
metadata: { name: web }
spec:
  selector: { app: web }
  ports:
    - { port: 80, targetPort: 80 }
```

Output

```
service/web created
kubectl get svc web
NAME  TYPE        CLUSTER-IP     PORT(S)
web   ClusterIP   10.96.12.7     80/TCP
```

Explanation

- **selector: app=web** tells the Service which pods to route to.
- The **ClusterIP** is a stable virtual IP inside the cluster.
- Traffic to the Service is load-balanced across matching pods.

> **Key idea:** Services solve the 'pod IPs keep changing' problem with a stable virtual IP and DNS.

---

031. Understand port vs targetPort

Scenario — Your Service listens on 80 but the app inside the pod listens on 8080.

Concept — port is what the Service exposes; targetPort is the container port it forwards to. They can differ, decoupling the service-facing port from the app's port.

`svc.yaml`

```yaml
spec:
  selector: { app: api }
  ports:
    - { port: 80, targetPort: 8080 }
```

Output

```
# clients hit web:80, traffic forwards to container:8080
kubectl get endpoints api
api   10.244.1.7:8080,10.244.2.3:8080
```

Explanation

- **port: 80** is the address clients use on the Service.
- **targetPort: 8080** is where the pod's container actually listens.
- Endpoints show the real pod IPs and the target port.

> **Key idea:** port = Service side; targetPort = container side. They needn't match.

032. Reach a Service by DNS name

Scenario — Your app needs to call another service by name, not IP.

Concept — Kubernetes runs cluster DNS (CoreDNS). Every Service gets a DNS record like name.namespace.svc.cluster.local, so pods reach services by stable names.

Command / Code

```
kubectl run tmp --rm -it --image=busybox --restart=Never -- \
  wget -qO- http://web.nimbus.svc.cluster.local | head -1
```

Output

```
<!DOCTYPE html>
# resolved web.nimbus.svc.cluster.local to the Service ClusterIP
```

Explanation

- Cluster DNS resolves **service.namespace.svc.cluster.local**.
- Within the same namespace you can just use the short name **web**.
- This is how microservices find each other without hardcoded IPs.

> **Key idea:** Same-namespace calls can use the short Service name; cross-namespace needs the full DNS.

---

033. Check a Service's endpoints

Scenario — A Service returns no response and you suspect it isn't matching any pods.

Concept — Endpoints are the actual pod IPs a Service routes to, derived from its selector. Empty endpoints mean the selector matches nothing — the classic 'Service has no endpoints' bug.

Command / Code

```
kubectl get endpoints web
kubectl get pods -l app=web -o wide
```

Output

```
NAME   ENDPOINTS
web    <none>
# no pods match app=web — fix the label or selector
```

Explanation

- **get endpoints** shows the pod IPs behind a Service.
- means the selector matched zero ready pods.
- Compare the Service selector with the pods' actual labels.

> **Key idea:** 'Service has no endpoints' almost always means a label/selector mismatch.

---

034. Expose a Service externally with NodePort

Scenario — You need to reach a service from outside the cluster in a bare setup without a cloud LB.

Concept — A NodePort opens a fixed high port on every node that forwards to the Service. It's a simple external entry point, mainly for dev or on-prem clusters.

`svc-nodeport.yaml`

```yaml
spec:
  type: NodePort
  selector: { app: web }
  ports:
    - { port: 80, targetPort: 80, nodePort: 30080 }
```

Output

```
service/web created
kubectl get svc web
web   NodePort   10.96.12.7   80:30080/TCP
# reachable at http://<any-node-ip>:30080
```

Explanation

- **type: NodePort** opens the same port on every node.
- **nodePort: 30080** pins the external port (30000–32767 range).
- Traffic to any node IP on that port reaches the Service.

> **Key idea:** NodePort is fine for dev/on-prem; use LoadBalancer or Ingress for production.

---

035. Get a cloud load balancer with type LoadBalancer

Scenario — On a cloud provider you want a real external IP in front of your Service.

Concept — type: LoadBalancer asks the cloud provider to provision an external load balancer that routes to the Service. It's the standard way to expose services on managed clusters.

`svc-lb.yaml`

```yaml
spec:
  type: LoadBalancer
  selector: { app: web }
  ports: [ { port: 80, targetPort: 80 } ]
```

Output

```
kubectl get svc web
NAME  TYPE           EXTERNAL-IP     PORT(S)
web   LoadBalancer   <pending>        80:31xxx/TCP
web   LoadBalancer   34.120.10.5     80:31xxx/TCP
```

Explanation

- **type: LoadBalancer** triggers the cloud to create an external LB.
- **EXTERNAL-IP** shows until the LB is provisioned.
- Only works where a cloud controller implements it.

> **Key idea:** LoadBalancer needs a cloud provider; for many HTTP services, Ingress is cheaper.

---

036. Give StatefulSet pods stable DNS with a headless Service

Scenario — Each database replica needs its own stable network identity.

Concept — A headless Service (clusterIP: None) returns the pods' individual IPs instead of load-balancing. It's used with StatefulSets so each pod gets a stable DNS name.

`svc-headless.yaml`

```yaml
spec:
  clusterIP: None
  selector: { app: db }
  ports: [ { port: 5432 } ]
```

Output

```
kubectl get svc db
db   ClusterIP   None   5432/TCP
# db-0.db, db-1.db resolve to individual pod IPs
```

Explanation

- **clusterIP: None** makes the Service headless — no virtual IP.
- DNS returns each pod's IP, enabling per-pod addressing.
- Paired with a StatefulSet, pods get names like db-0.db.

> **Key idea:** Headless Services give StatefulSet pods stable, individual DNS names.

---

037. Forward a local port to a pod for debugging

Scenario — You want to hit a pod directly from your laptop without exposing it.

Concept — kubectl port-forward tunnels a local port to a pod or Service through the API server. It's a secure, temporary way to reach internal workloads during debugging.

Command / Code

```
kubectl port-forward deployment/web 8080:80
```

Output

```
Forwarding from 127.0.0.1:8080 -> 80
Forwarding from [::1]:8080 -> 80
# curl localhost:8080 now reaches the pod
```

Explanation

- **port-forward** tunnels localhost:8080 to the pod's port 80.
- It runs through the API server, so no Service or public IP is needed.
- Great for testing internal services privately from your machine.

> **Key idea:** port-forward is the safe way to poke an internal pod from your laptop.

---

038. Restrict traffic with a NetworkPolicy

Scenario — Only the frontend should be able to reach the backend pods.

Concept — By default all pods can talk to each other. A NetworkPolicy restricts traffic by pod selector, implementing least-privilege networking (requires a supporting CNI).

`netpol.yaml`

```
kind: NetworkPolicy
spec:
  podSelector: { matchLabels: { app: backend } }
  policyTypes: [ Ingress ]
  ingress:
    - from: [ { podSelector: { matchLabels: { app: frontend } } } ]
```

Output

```
networkpolicy.networking.k8s.io/backend-allow created
# frontend -> backend allowed
# any-other -> backend denied
```

Explanation

- **podSelector** chooses which pods the policy protects (backend).
- The **ingress from** rule allows only frontend pods in.
- Everything not explicitly allowed is denied once a policy selects a pod.

> **Key idea:** NetworkPolicies need a supporting CNI (Calico, Cilium) to take effect.

---

039. Alias an external service with ExternalName

Scenario — You want pods to reach an external database by an internal-looking name.

Concept — An ExternalName Service maps a cluster DNS name to an external hostname via a CNAME. It lets in-cluster clients use a stable internal name for an outside dependency.

`svc-externalname.yaml`

```
kind: Service
metadata: { name: db }
spec:
  type: ExternalName
  externalName: db.prod.example.com
```

Output

```
service/db created
# db.nimbus.svc.cluster.local -> CNAME db.prod.example.com
```

Explanation

- **type: ExternalName** creates a DNS CNAME, not a proxy.
- Pods calling **db** are redirected to the external hostname.
- No selectors or endpoints are involved — it's pure DNS.

> **Key idea:** ExternalName gives an external dependency a stable in-cluster name.

ConfigMaps & Secrets

040. Create a ConfigMap from literals

Scenario — Your app needs non-secret settings you'd rather not bake into the image.

Concept — A ConfigMap stores non-confidential configuration as key/value pairs, decoupling config from images. Pods consume it as env vars or files.

Command / Code

```
kubectl create configmap app-config \
  --from-literal=LOG_LEVEL=info \
  --from-literal=APP_ENV=prod
kubectl get configmap app-config -o jsonpath='{.data}'
```

Output

```
configmap/app-config created
{"APP_ENV":"prod","LOG_LEVEL":"info"}
```

Explanation

- **--from-literal** adds key/value pairs directly.
- The ConfigMap keeps config outside the container image.
- One ConfigMap can be shared by many pods.

> **Key idea:** ConfigMaps decouple config from images — change config without rebuilding.

---

041. Inject a ConfigMap as environment variables

Scenario — You want all keys from a ConfigMap available as env vars in a container.

Concept — envFrom injects every key of a ConfigMap as environment variables at once, avoiding a long, repetitive env list.

`pod-envfrom.yaml`

```
spec:
  containers:
    - name: app
      image: nimbus-app:1.0
      envFrom:
        - configMapRef: { name: app-config }
```

Output

```
kubectl exec app -- printenv LOG_LEVEL APP_ENV
info
prod
```

Explanation

- **envFrom + configMapRef** imports all keys as env vars.
- Each ConfigMap key becomes a variable of the same name.
- Use **env + configMapKeyRef** to import a single key instead.

> **Key idea:** envFrom imports a whole ConfigMap; configMapKeyRef imports one key.

---

042. Mount a ConfigMap as files

Scenario — Your app reads a config file from disk, not environment variables.

Concept — A ConfigMap can be mounted as a volume, turning each key into a file. This suits apps that expect a config file path.

`pod-cmvol.yaml`

```
spec:
  containers:
    - name: app
      image: nimbus-app:1.0
      volumeMounts: [ { name: cfg, mountPath: /etc/app } ]
  volumes:
    - name: cfg
      configMap: { name: app-config }
```

Output

```
kubectl exec app -- ls /etc/app
APP_ENV
LOG_LEVEL
# each key became a file under /etc/app
```

Explanation

- Mounting a ConfigMap as a volume creates one file per key.
- The file's contents are the key's value.
- Mounted ConfigMap updates propagate to the files (with a short delay).

> **Key idea:** Mount ConfigMaps as files for apps that read config from a path.

---

043. Create a Secret for sensitive data

Scenario — Your app needs a database password that must not sit in plain manifests.

Concept — A Secret holds sensitive data, base64-encoded and handled more carefully than a ConfigMap. It's the right place for passwords, tokens, and keys.

Command / Code

```
kubectl create secret generic db-cred \
  --from-literal=username=nimbus \
  --from-literal=password=s3cr3t
kubectl get secret db-cred -o jsonpath='{.data.password}' | base64 -d
```

Output

```
secret/db-cred created
s3cr3t
# stored base64-encoded; decode to read
```

Explanation

- **create secret generic** stores key/value pairs as a Secret.
- Values are base64-encoded (encoding, not encryption, by itself).
- Enable encryption at rest and RBAC to truly protect Secrets.

> **Key idea:** Base64 is encoding, not security — pair Secrets with RBAC and encryption at rest.

---

044. Consume a Secret as environment variables

Scenario — Your container reads the DB credentials from environment variables.

Concept — Secrets can be injected as env vars via secretKeyRef, keeping credentials out of the image and the pod spec's plaintext.

`pod-secretenv.yaml`

```
      env:
        - name: DB_USER
          valueFrom: { secretKeyRef: { name: db-cred, key: username } }
        - name: DB_PASS
          valueFrom: { secretKeyRef: { name: db-cred, key: password } }
```

Output

```
kubectl exec app -- printenv DB_USER
nimbus
# DB_PASS present but not shown here
```

Explanation

- **secretKeyRef** pulls a single key from a Secret into an env var.
- The value isn't written into the pod manifest in plaintext.
- Prefer mounting as files for very sensitive values (see next).

> **Key idea:** secretKeyRef injects one Secret key; envFrom + secretRef injects all keys.

---

045. Mount a Secret as files

Scenario — A TLS cert and key must appear as files for your server to load.

Concept — Mounting a Secret as a volume writes each key to a file, often in-memory (tmpfs). This is preferred for certificates and keys that apps read from disk.

`pod-secretvol.yaml`

```
      volumeMounts: [ { name: tls, mountPath: /etc/tls, readOnly: true } ]
  volumes:
    - name: tls
      secret: { secretName: web-tls }
```

Output

```
kubectl exec app -- ls /etc/tls
tls.crt
tls.key
# backed by tmpfs (in-memory), read-only
```

Explanation

- Each Secret key becomes a file under the mount path.
- Secret volumes are typically backed by **tmpfs** (memory, not disk).
- **readOnly: true** prevents the app from modifying them.

> **Key idea:** Mount certs/keys as Secret volumes — they live in memory, not on disk.

---

046. Build a ConfigMap from a file

Scenario — You already have an nginx.conf and want it available in the cluster.

Concept — --from-file loads a whole file into a ConfigMap under a key named after the file, letting you manage existing config files declaratively.

Command / Code

```
kubectl create configmap nginx-conf --from-file=nginx.conf
kubectl get configmap nginx-conf -o jsonpath='{.data}' | head -c 60
```

Output

```
configmap/nginx-conf created
{"nginx.conf":"user nginx;\nworker_processes auto;
```

Explanation

- **--from-file** stores the file's contents under a matching key.
- Mount this ConfigMap to place nginx.conf into the container.
- Pass a directory to load multiple files at once.

> **Key idea:** --from-file turns existing config files into ConfigMaps you can mount.

---

047. Prevent accidental config changes with immutability

Scenario — A critical ConfigMap must not be edited in place once deployed.

Concept — An immutable ConfigMap or Secret cannot be updated after creation, only replaced. This prevents accidental live edits and reduces API server watch load.

`configmap-immutable.yaml`

```
apiVersion: v1
kind: ConfigMap
metadata: { name: app-config }
immutable: true
data: { LOG_LEVEL: info }
```

Output

```
configmap/app-config created
kubectl patch cm app-config -p '{"data":{"LOG_LEVEL":"debug"}}'
Error: field is immutable
```

Explanation

- **immutable: true** locks the object's data after creation.
- Updates are rejected; you must delete and recreate to change it.
- This guards critical config and improves cluster performance.

> **Key idea:** Mark stable config immutable to block accidental edits and cut watch overhead.

---

048. Roll pods when config changes

Scenario — You updated a ConfigMap used as env vars, but pods still use the old values.

Concept — ConfigMap values injected as env vars are read only at pod start; they don't update live. To pick up changes you must trigger a rollout of the consuming pods.

Command / Code

```
kubectl create configmap app-config --from-literal=LOG_LEVEL=debug \
  --dry-run=client -o yaml | kubectl apply -f -
kubectl rollout restart deployment/web
```

Output

```
configmap/app-config configured
deployment.apps/web restarted
# new pods start with the updated env value
```

Explanation

- Env vars from a ConfigMap are captured at pod startup only.
- **rollout restart** recreates pods so they read the new value.
- Mounted ConfigMap files do update live, but env vars do not.

> **Key idea:** After changing env-based config, rollout restart the Deployment to apply it.

Storage: Volumes, PV & PVC

049. Share scratch space with emptyDir

Scenario — Two containers in a pod need a shared temporary directory.

Concept — An emptyDir volume is created when a pod starts and deleted when it stops. It's ephemeral scratch space shared by the pod's containers.

`pod-emptydir.yaml`

```
spec:
  containers:
    - name: writer
      image: busybox
      command: ['sh','-c','echo hi > /data/f; sleep 3600']
      volumeMounts: [ { name: scratch, mountPath: /data } ]
  volumes: [ { name: scratch, emptyDir: {} } ]
```

Output

```
kubectl exec app -c writer -- cat /data/f
hi
# gone when the pod is deleted
```

Explanation

- **emptyDir** gives the pod a fresh, shared directory.
- All containers in the pod can mount and share it.
- Its lifetime is the pod's — data is lost when the pod dies.

> **Key idea:** emptyDir is ephemeral pod-scoped scratch space, not durable storage.

---

050. Request durable storage with a PVC

Scenario — Your database must keep its data even if the pod is rescheduled.

Concept — A PersistentVolumeClaim requests storage of a given size and access mode. A StorageClass provisions a matching PersistentVolume dynamically, giving pods durable storage.

`pvc.yaml`

```
kind: PersistentVolumeClaim
metadata: { name: pgdata }
spec:
  accessModes: [ ReadWriteOnce ]
  resources: { requests: { storage: 10Gi } }
  storageClassName: standard
```

Output

```
persistentvolumeclaim/pgdata created
kubectl get pvc pgdata
NAME     STATUS   VOLUME       CAPACITY   ACCESS
pgdata   Bound    pvc-9f2c...  10Gi       RWO
```

Explanation

- The PVC requests **10Gi** with **ReadWriteOnce** access.
- The StorageClass provisions a PV and the PVC binds to it (**Bound**).
- Pods reference the PVC, not the underlying PV, for portability.

> **Key idea:** Pods claim storage via PVCs; StorageClasses provision the real volumes dynamically.

---

051. Mount a PVC into a pod

Scenario — Your Postgres container needs its data directory backed by the PVC.

Concept — A pod mounts a PVC like any volume, and the data survives pod restarts and rescheduling because it lives on the bound PersistentVolume.

`pod-pvc.yaml`

```yaml
spec:
  containers:
    - name: db
      image: postgres:16
      volumeMounts: [ { name: data, mountPath: /var/lib/postgresql/data } ]
  volumes:
    - name: data
      persistentVolumeClaim: { claimName: pgdata }
```

Output

```
pod/db created
kubectl exec db -- df -h /var/lib/postgresql/data
/dev/sdb   10G   40M   9.9G   1% /var/lib/postgresql/data
```

Explanation

- The pod mounts the PVC by **claimName**.
- Data written there persists on the underlying PV.
- If the pod is rescheduled, it reattaches the same storage.

> **Key idea:** Mounting a PVC gives a pod storage that outlives the pod itself.

---

052. Understand access modes

Scenario — You need to know whether multiple pods can write to one volume.

Concept — Access modes define how a volume can be mounted: ReadWriteOnce (one node), ReadOnlyMany, and ReadWriteMany (many nodes). Support depends on the storage backend.

Command / Code

```
# RWO  : one node can mount read-write (most block storage)
# ROX  : many nodes read-only
# RWX  : many nodes read-write (NFS, CephFS, some CSI)
```

Output

```
# block volumes (EBS, PD) are typically RWO
# shared filesystems (NFS) can be RWX
```

Explanation

- **ReadWriteOnce** allows one node to mount it read-write.
- **ReadWriteMany** allows many nodes — needs a shared filesystem.
- Requesting a mode the backend can't provide leaves the PVC Pending.

> **Key idea:** Most block storage is RWO; you need NFS/CephFS-style backends for RWX.

---

053. Control cleanup with reclaim policy

Scenario — You deleted a PVC and want to know what happens to the data.

Concept — A PV's reclaim policy decides its fate when its PVC is deleted: Delete removes the underlying volume, Retain keeps it for manual recovery.

Command / Code

```
kubectl get pv -o custom-
columns=NAME:.metadata.name,POLICY:.spec.persistentVolumeReclaimPolicy
```

Output

```
NAME          POLICY
pvc-9f2c...   Delete
# Delete: data is destroyed when the PVC is removed
```

Explanation

- **Delete** (common default) destroys the volume with the PVC.
- **Retain** keeps the PV and data for manual recovery.
- Set Retain for critical data so a deleted PVC doesn't lose it.

> **Key idea:** Use Retain for critical data so deleting a PVC doesn't erase the volume.

---

054. Provision storage dynamically with a StorageClass

Scenario — You don't want to pre-create PersistentVolumes by hand.

Concept — A StorageClass describes a provisioner and parameters. When a PVC references it, Kubernetes creates a matching PV on demand — dynamic provisioning.

Command / Code

```
kubectl get storageclass
```

Output

```
NAME                 PROVISIONER            RECLAIMPOLICY
standard (default)   ebs.csi.aws.com        Delete
fast-ssd             ebs.csi.aws.com        Delete
```

Explanation

- Each **StorageClass** maps to a provisioner (a CSI driver).
- A PVC referencing it triggers automatic PV creation.
- The **(default)** class is used when a PVC names none.

> **Key idea:** Dynamic provisioning via StorageClass means you never hand-create PVs.

---

055. Expand a PersistentVolumeClaim

Scenario — A volume is filling up and you need more space without recreating it.

Concept — If the StorageClass allows volume expansion, you can increase a PVC's requested size and the backend grows the volume in place.

Command / Code

```
kubectl patch pvc pgdata -p '{"spec":{"resources":{"requests":{"storage":"20Gi"}}}}'
kubectl get pvc pgdata
```

Output

```
persistentvolumeclaim/pgdata patched
NAME     STATUS   CAPACITY
pgdata   Bound    20Gi
```

Explanation

- Patching the PVC's requested storage triggers expansion.
- This requires **allowVolumeExpansion: true** on the StorageClass.

- You can only grow a PVC, never shrink it.

> **Key idea:** PVCs can grow (if the StorageClass allows it) but never shrink.

---

056. Avoid hostPath in multi-node clusters

Scenario — A pod using hostPath breaks when it's rescheduled to another node.

Concept — hostPath mounts a directory from the node's filesystem. It ties a pod to one node's disk and is unsafe in multi-node clusters — use PVCs instead.

Command / Code

```
# fragile:
volumes:
  - name: data
    hostPath: { path: /mnt/data }
# node A's /mnt/data is NOT node B's /mnt/data
```

Output

```
# pod rescheduled to node B finds a different (empty) /mnt/data
```

Explanation

- **hostPath** reads/writes the specific node's local disk.
- Rescheduling to another node loses access to the original data.
- It also grants risky access to the node filesystem.

> **Key idea:** Avoid hostPath for app data in multi-node clusters — use PVCs.

---

057. Diagnose a PVC stuck in Pending

Scenario — You created a PVC but it never binds, so the pod using it stays Pending too.

Concept — A PVC stays Pending when no PV can satisfy it: no default StorageClass, an unknown class name, or a requested size/access mode the backend can't provide. The PVC events state the reason.

Command / Code

```
kubectl describe pvc pgdata | grep -A3 Events
```

Output

```
  Warning  ProvisioningFailed  persistentvolume-controller
  storageclass.storage.k8s.io "fast" not found
# fix: reference a real StorageClass or set a default
```

Explanation

- A Pending PVC blocks any pod that mounts it.
- The event names the exact cause — here a missing StorageClass.
- Fix by naming an existing class, setting a default, or adjusting the request.

> **Key idea:** A Pending PVC almost always means a missing/unknown StorageClass — read its events.

Scheduling & Resources

058. Pin pods to nodes with nodeSelector

Scenario — A GPU workload must only run on nodes that actually have GPUs.

Concept — nodeSelector is the simplest scheduling constraint: a pod runs only on nodes whose labels match. It's a hard requirement, not a preference.

`pod-nodeselector.yaml`

```
spec:
  nodeSelector: { hardware: gpu }
  containers: [ { name: train, image: nimbus-ml:1.0 } ]
```

Output

```
kubectl label node worker-3 hardware=gpu
node/worker-3 labeled
kubectl get pod train -o wide
train  1/1  Running  worker-3
```

Explanation

- **nodeSelector** requires the node to carry the matching label.
- The pod stays **Pending** until a labelled node is available.
- For richer rules (preferences, operators), use affinity instead.

> **Key idea:** nodeSelector is a hard label match; affinity adds soft rules and operators.

---

059. Express flexible placement with node affinity

Scenario — You prefer SSD nodes but can tolerate others if none are free.

Concept — nodeAffinity supports required (hard) and preferred (soft) rules with operators like In and NotIn, giving far more expressive placement than nodeSelector.

`pod-affinity.yaml`

```
  affinity:
    nodeAffinity:
      preferredDuringSchedulingIgnoredDuringExecution:
        - weight: 100
          preference:
            matchExpressions: [ { key: disk, operator: In, values: [ ssd ] } ]
```

Output

```
# scheduler prefers ssd nodes, but will use others if needed
kubectl get pod app -o wide
app  1/1  Running  worker-2 (ssd)
```

Explanation

- **preferred...** is a soft rule with a weight, not a hard requirement.
- The scheduler favours matching nodes but won't leave the pod Pending.
- Use **required...** for hard constraints instead.

> **Key idea:** Affinity 'preferred' = soft preference; 'required' = hard rule.

---

060. Spread or co-locate pods with pod affinity

Scenario — You want replicas of a service on different nodes for resilience.

Concept — Pod anti-affinity keeps pods apart (spread for availability); pod affinity draws them together (co-locate for latency). Both use a topologyKey like hostname or zone.

`pod-antiaffinity.yaml`

```
  affinity:
    podAntiAffinity:
      requiredDuringSchedulingIgnoredDuringExecution:
        - labelSelector: { matchLabels: { app: web } }
          topologyKey: kubernetes.io/hostname
```

Output

```
# two web pods refuse to share a node
web-1  worker-1
web-2  worker-2
```

Explanation

- **podAntiAffinity** stops matching pods sharing the same topology domain.
- **topologyKey: hostname** means 'not the same node'.
- This spreads replicas so one node failure doesn't take them all.

> **Key idea:** Anti-affinity spreads replicas across nodes/zones for resilience.

---

061. Reserve nodes with taints and tolerations

Scenario — Dedicated nodes should run only special workloads, not general pods.

Concept — A taint repels pods from a node unless they carry a matching toleration. Together they reserve nodes for specific workloads.

Command / Code

```
kubectl taint nodes worker-4 dedicated=ml:NoSchedule
# pod must tolerate it:
  tolerations:
    - { key: dedicated, operator: Equal, value: ml, effect: NoSchedule }
```

Output

```
node/worker-4 tainted
# only pods tolerating dedicated=ml land on worker-4
```

Explanation

- The **taint** repels pods lacking the matching toleration.
- **NoSchedule** blocks new pods; existing ones stay.
- A pod's **toleration** lets it be scheduled there.

> **Key idea:** Taints repel; tolerations permit — together they reserve nodes.

---

062. Drain a node for maintenance

Scenario — You need to patch a node and must move its pods off first.

Concept — cordon marks a node unschedulable; drain evicts its pods (respecting disruption budgets) so they reschedule elsewhere before maintenance.

Command / Code

```
kubectl cordon worker-1
kubectl drain worker-1 --ignore-daemonsets --delete-emptydir-data
```

Output

```
node/worker-1 cordoned
evicting pod nimbus/web-1
node/worker-1 drained
# pods rescheduled to other nodes
```

Explanation

- **cordon** stops new pods from scheduling onto the node.
- **drain** evicts existing pods so they move elsewhere.

- **--ignore-daemonsets** skips per-node DaemonSet pods that can't move.

> **Key idea:** cordon + drain safely empties a node before maintenance; uncordon to restore it.

---

063. Cap a namespace with ResourceQuota

Scenario — One team's namespace keeps consuming the whole cluster's resources.

Concept — A ResourceQuota limits aggregate resource use (CPU, memory, object counts) in a namespace, enforcing fair sharing on a multi-tenant cluster.

`quota.yaml`

```yaml
kind: ResourceQuota
metadata: { name: team-quota }
spec:
  hard:
    requests.cpu: '4'
    requests.memory: 8Gi
    pods: '20'
```

Output

```
resourcequota/team-quota created
# creating a pod beyond the quota is rejected
Error: exceeded quota: requests.cpu
```

Explanation

- **hard** sets the ceiling for the namespace as a whole.
- New pods that would exceed it are rejected at creation.
- Quotas require pods to declare requests to be counted.

> **Key idea:** ResourceQuotas enforce fair sharing per namespace on shared clusters.

---

064. Set defaults with a LimitRange

Scenario — Pods without resource requests slip through and break scheduling fairness.

Concept — A LimitRange sets default and maximum resource values for containers in a namespace, ensuring every pod has sensible requests and limits even if authors forget.

`limitrange.yaml`

```yaml
kind: LimitRange
metadata: { name: defaults }
spec:
  limits:
    - type: Container
      default: { cpu: 500m, memory: 256Mi }
      defaultRequest: { cpu: 100m, memory: 128Mi }
```

Output

```
limitrange/defaults created
# pods without resources now get these defaults applied
```

Explanation

- **default** supplies limits when a container omits them.
- **defaultRequest** supplies requests similarly.
- This guarantees scheduling and quota accounting work correctly.

> **Key idea:** LimitRange backfills requests/limits so no pod escapes resource accounting.

---

065. Prioritise critical pods with PriorityClass

Scenario — Under resource pressure, critical pods must win over batch jobs.

Concept — A PriorityClass assigns importance to pods. When resources are scarce, the scheduler can preempt (evict) lower-priority pods to make room for higher-priority ones.

`priority.yaml`

```yaml
kind: PriorityClass
metadata: { name: high }
value: 100000
globalDefault: false
# pod: spec.priorityClassName: high
```

Output

```
priorityclass.scheduling.k8s.io/high created
# high-priority pod preempts a low-priority one when nodes are full
```

Explanation

- **value** ranks the class; higher wins.
- The scheduler may **preempt** lower-priority pods to schedule higher ones.
- Reserve high priorities for genuinely critical workloads.

> **Key idea:** PriorityClasses let critical pods preempt less important ones under pressure.

---

066. Balance pods across zones with spread constraints

Scenario — Replicas cluster in one availability zone, risking a zone-wide outage.

Concept — topologySpreadConstraints evenly distribute pods across topology domains (zones, nodes) within a skew you set, improving resilience without strict anti-affinity.

`pod-spread.yaml`

```
  topologySpreadConstraints:
    - maxSkew: 1
      topologyKey: topology.kubernetes.io/zone
      whenUnsatisfiable: DoNotSchedule
      labelSelector: { matchLabels: { app: web } }
```

Output

```
# pods balanced: zone-a 2, zone-b 2, zone-c 2 (skew <= 1)
```

Explanation

- **maxSkew: 1** keeps zone counts within one of each other.
- **whenUnsatisfiable: DoNotSchedule** makes it a hard rule.
- This spreads replicas across zones for fault tolerance.

> **Key idea:** Spread constraints balance replicas across zones/nodes to survive failures.

---

067. Diagnose a Pending pod

Scenario — A pod stays Pending and never gets scheduled.

Concept — Pending usually means the scheduler can't place the pod: insufficient resources, an unsatisfiable selector/affinity, or taints. The pod's events explain exactly why.

Command / Code

```
kubectl describe pod app | grep -A5 Events
```

Output

```
Events:
  Warning  FailedScheduling  default-scheduler
  0/3 nodes are available: 3 Insufficient cpu.
```

Explanation

- **FailedScheduling** events state the scheduling obstacle.
- Here no node has enough free CPU for the pod's requests.
- Fix by lowering requests, adding capacity, or relaxing constraints.

> **Key idea:** For a Pending pod, read the FailedScheduling event — it names the exact reason.

Health, Probes & Autoscaling

068. Restart unhealthy containers with a liveness probe

Scenario — An app process hangs but the container stays 'running', so traffic keeps failing.

Concept — A liveness probe tells the kubelet whether a container is healthy. On repeated failure the kubelet restarts the container, recovering from hangs and deadlocks.

`pod-liveness.yaml`

```
    livenessProbe:
      httpGet: { path: /healthz, port: 8080 }
      initialDelaySeconds: 10
      periodSeconds: 10
      failureThreshold: 3
```

Output

```
kubectl get pod app
app  1/1  Running  2  6m
# RESTARTS incremented after liveness failures
```

Explanation

- The probe periodically checks **/healthz**.
- After **failureThreshold** failures, the kubelet restarts the container.
- This recovers hung processes that would otherwise stay 'running'.

> **Key idea:** Liveness probes restart hung containers — keep the check cheap and specific.

---

069. Gate traffic with a readiness probe

Scenario — New pods receive traffic before they've finished warming up, causing errors.

Concept — A readiness probe decides whether a pod should receive traffic. Failing pods are removed from Service endpoints without being restarted — ideal during startup or overload.

`pod-readiness.yaml`

```
    readinessProbe:
      httpGet: { path: /ready, port: 8080 }
      periodSeconds: 5
```

Output

```
kubectl get pod app
app  0/1  Running  0  8s
app  1/1  Running  0  20s  (now in Service endpoints)
```

Explanation

- Until **/ready** passes, the pod shows **0/1** ready.
- Not-ready pods are excluded from the Service's endpoints.
- Unlike liveness, a failing readiness probe does not restart the pod.

> **Key idea:** Readiness gates traffic; liveness restarts. Use both for smooth rollouts.

---

070. Protect slow starters with a startup probe

Scenario — A slow-booting app is killed by its liveness probe before it finishes starting.

Concept — A startup probe runs first and disables liveness/readiness until it succeeds, giving slow apps time to initialise without being killed prematurely.

`pod-startup.yaml`

```yaml
    startupProbe:
      httpGet: { path: /healthz, port: 8080 }
      failureThreshold: 30
      periodSeconds: 10
```

Output

```
# up to 300s allowed to start before liveness kicks in
app  1/1  Running  0  4m
```

Explanation

- The **startupProbe** must pass before liveness/readiness begin.
- **failureThreshold × periodSeconds** sets the startup budget (here 300s).
- This prevents liveness from killing a still-initialising app.

> **Key idea:** Add a startupProbe for slow-booting apps so liveness doesn't kill them early.

---

071. Choose the right probe type

Scenario — Your app has no HTTP endpoint but you still need a health check.

Concept — Probes support three mechanisms: httpGet (an HTTP endpoint), tcpSocket (a port opens), and exec (a command exits 0). Pick the one that reflects real health.

Command / Code

```
# HTTP:  httpGet: { path: /healthz, port: 8080 }
# TCP:   tcpSocket: { port: 5432 }
# EXEC:  exec: { command: ['sh','-c','pg_isready -U postgres'] }
```

Output

```
# pick the check that truly reflects app health
```

Explanation

- **httpGet** suits web apps with a health route.
- **tcpSocket** just checks a port is accepting connections.
- **exec** runs a command — good for databases and non-HTTP apps.

> **Key idea:** Match the probe type to the app: http for web, tcp for ports, exec for CLIs.

---

072. Autoscale on CPU with an HPA

Scenario — Traffic spikes and you want replicas to scale automatically with load.

Concept — A HorizontalPodAutoscaler adjusts a Deployment's replica count to keep a metric (like CPU) near a target. It needs resource requests and the metrics-server.

Command / Code

```
kubectl autoscale deployment web --cpu-percent=60 --min=2 --max=10
kubectl get hpa web
```

Output

```
horizontalpodautoscaler.autoscaling/web autoscaled
NAME   REFERENCE         TARGETS   MINPODS   MAXPODS   REPLICAS
web    Deployment/web   35%/60%   2         10        2
```

Explanation

- The HPA scales to keep average CPU near **60%** of requests.
- It stays within **--min** and **--max** bounds.
- It requires CPU **requests** set and a running metrics-server.

> **Key idea:** HPA needs resource requests + metrics-server to compute utilisation.

---

073. Watch an HPA react to load

Scenario — You want to confirm the autoscaler actually adds pods under pressure.

Concept — The HPA controller samples metrics periodically and changes replicas gradually, with stabilisation to avoid flapping. Watching it shows the scaling decisions live.

Command / Code

```
kubectl get hpa web --watch
```

Output

```
web  Deployment/web  35%/60%   2  10  2
web  Deployment/web  88%/60%  2  10  2
web  Deployment/web  72%/60%   2  10  4
```

Explanation

- Rising utilisation above target prompts the HPA to add replicas.
- It scales up quickly but down slowly to avoid flapping.
- Replica changes lag metrics by the controller's sync period.

> **Key idea:** HPA scales up fast and down slow — stabilisation prevents thrashing.

---

074. Protect availability with a PodDisruptionBudget

Scenario — A node drain could take down too many replicas of a critical service at once.

Concept — A PodDisruptionBudget limits how many pods of a set can be voluntarily disrupted (drains, upgrades) at once, protecting availability during maintenance.

`pdb.yaml`

```
kind: PodDisruptionBudget
metadata: { name: web-pdb }
spec:
  minAvailable: 2
  selector: { matchLabels: { app: web } }
```

Output

```
poddisruptionbudget.policy/web-pdb created
# drain will keep at least 2 web pods running at all times
```

Explanation

- **minAvailable: 2** keeps at least two pods up during disruptions.
- Drains and rolling node upgrades respect the PDB.
- It guards against voluntary disruptions, not crashes.

> **Key idea:** PDBs cap voluntary disruptions (drains/upgrades) to protect availability.

---

075. See why a probe is failing

Scenario — A pod keeps restarting and you suspect the liveness probe is misconfigured.

Concept — Probe failures surface as Warning events on the pod. Reading them tells you whether the endpoint, port, or timing is wrong before you tweak the app.

Command / Code

```
kubectl describe pod app | grep -iA2 probe
```

Output

```
  Warning  Unhealthy  kubelet  Liveness probe failed: HTTP probe failed
           with statuscode: 500
```

Explanation

- **Unhealthy** events report the failing probe and reason.
- A 500 here means the app's health endpoint itself is erroring.
- Misconfigured port/path or too-tight timing also show up here.

> **Key idea:** Probe failures appear as Unhealthy events — read them before blaming the app.

---

076. Autoscale on custom or memory metrics

Scenario — CPU isn't the right signal; you need to scale on memory or a custom metric.

Concept — The autoscaling/v2 HPA supports multiple metric types — resource (CPU/memory), pods, and external/custom — letting you scale on whatever truly reflects load.

`hpa-v2.yaml`

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
spec:
  scaleTargetRef: { kind: Deployment, name: web }
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource: { name: memory, target: { type: Utilization, averageUtilization: 70 } }
```

Output

```
horizontalpodautoscaler.autoscaling/web created
web  ...  memory: 55%/70%  2  10  2
```

Explanation

- **autoscaling/v2** allows metrics beyond CPU.

- Here it targets 70% average **memory** utilisation.
- Custom/external metrics need an adapter (e.g. Prometheus adapter).

> **Key idea:** Use autoscaling/v2 to scale on memory or custom metrics, not just CPU.

RBAC & Security

077. Give a workload an identity with a ServiceAccount

Scenario — A pod needs to call the Kubernetes API and should not use the default identity.

Concept — Every pod runs as a ServiceAccount, its identity to the API server. Creating a dedicated one lets you grant least-privilege permissions to that workload.

Command / Code

```
kubectl create serviceaccount nimbus-api
# in the pod spec:
  serviceAccountName: nimbus-api
```

Output

```
serviceaccount/nimbus-api created
# pod now authenticates to the API as nimbus-api
```

Explanation

- A **ServiceAccount** is the pod's identity to the API server.
- Assigning a dedicated SA avoids over-permissioning via the default one.
- Permissions are granted separately through RBAC (next scenarios).

> **Key idea:** Give each workload its own ServiceAccount so you can scope its permissions.

---

078. Grant namespaced permissions with Role + RoleBinding

Scenario — A service account should read pods in its namespace, nothing more.

Concept — A Role defines permissions within a namespace; a RoleBinding grants that Role to a subject (user or ServiceAccount). Together they implement least privilege.

`rbac.yaml`

```
kind: Role
metadata: { name: pod-reader }
rules:
  - apiGroups: ['']
    resources: [pods]
    verbs: [get, list, watch]
---
kind: RoleBinding
subjects: [ { kind: ServiceAccount, name: nimbus-api } ]
roleRef: { kind: Role, name: pod-reader }
```

Output

```
role.rbac.authorization.k8s.io/pod-reader created
rolebinding.rbac.authorization.k8s.io/... created
```

Explanation

- The **Role** lists allowed verbs on specific resources.
- The **RoleBinding** ties that Role to the ServiceAccount.
- Scope stays within one namespace — exactly least privilege.

> **Key idea:** Role + RoleBinding = namespaced least-privilege access.

---

079. Grant cluster-wide access with ClusterRole

Scenario — A monitoring agent needs to read nodes, which are cluster-scoped.

Concept — ClusterRole and ClusterRoleBinding work like Role/RoleBinding but span the whole cluster, needed for cluster-scoped resources like nodes or for access across all namespaces.

`rbac-cluster.yaml`

```
kind: ClusterRole
metadata: { name: node-reader }
rules:
  - apiGroups: ['']
    resources: [nodes]
    verbs: [get, list]
---
kind: ClusterRoleBinding
subjects: [ { kind: ServiceAccount, name: monitor, namespace: obs } ]
roleRef: { kind: ClusterRole, name: node-reader }
```

Output

```
clusterrole.rbac.authorization.k8s.io/node-reader created
clusterrolebinding.rbac.authorization.k8s.io/... created
```

Explanation

- **ClusterRole** covers cluster-scoped resources like nodes.
- A **ClusterRoleBinding** grants it across the whole cluster.
- Bind a ClusterRole with a RoleBinding to limit it to one namespace.

> **Key idea:** Use ClusterRoles for cluster-scoped resources or cross-namespace access.

---

080. Check what a subject can do

Scenario — You need to verify a ServiceAccount's permissions before trusting it.

Concept — kubectl auth can-i answers whether a verb is allowed, optionally impersonating a subject. It's the quickest way to test RBAC without deploying anything.

Command / Code

```
kubectl auth can-i list pods \
  --as=system:serviceaccount:nimbus:nimbus-api
kubectl auth can-i delete nodes --as=system:serviceaccount:nimbus:nimbus-api
```

Output

```
yes
no
```

Explanation

- **auth can-i** evaluates RBAC for a verb/resource.
- **--as** impersonates the subject you want to test.
- This confirms least privilege without trial-and-error deploys.

> **Key idea:** kubectl auth can-i --as ... is the fastest way to verify RBAC.

---

081. Run containers as non-root

Scenario — A security review requires that no container runs as root.

Concept — A securityContext sets runtime security options like runAsNonRoot and a specific UID, reducing the impact of a container compromise.

`pod-securitycontext.yaml`

```
spec:
  securityContext: { runAsNonRoot: true, runAsUser: 1000, fsGroup: 2000 }
  containers:
    - name: app
      image: nimbus-app:1.0
```

Output

```
kubectl exec app -- id
uid=1000 gid=0 groups=2000
# pod refuses to start if the image would run as root
```

Explanation

- **runAsNonRoot: true** blocks the pod if it would run as root.
- **runAsUser** pins a specific non-root UID.
- **fsGroup** sets group ownership for mounted volumes.

> **Key idea:** runAsNonRoot + a fixed UID contains the blast radius of a compromise.

---

082. Harden a container's capabilities and filesystem

Scenario — You want to drop Linux capabilities and make the root filesystem read-only.

Concept — Per-container securityContext can drop all capabilities, forbid privilege escalation, and enforce a read-only root filesystem — strong, low-effort hardening.

`pod-hardened.yaml`

```
      securityContext:
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
        capabilities: { drop: [ ALL ] }
```

Output

```
# no capabilities, no escalation, immutable rootfs
# mount an emptyDir for any writable paths the app needs
```

Explanation

- **drop: [ALL]** removes every Linux capability.
- **readOnlyRootFilesystem** blocks tampering and persistence.
- **allowPrivilegeEscalation: false** stops gaining new privileges.

> **Key idea:** Drop ALL caps, forbid escalation, and use a read-only rootfs by default.

---

083. Enforce Pod Security Standards

Scenario — You want the cluster to reject privileged pods in a namespace automatically.

Concept — Pod Security Admission enforces the Pod Security Standards (privileged, baseline, restricted) per namespace via labels, blocking non-compliant pods at admission.

Command / Code

```
kubectl label namespace nimbus \
  pod-security.kubernetes.io/enforce=restricted
```

Output

```
namespace/nimbus labeled
# a privileged pod is now rejected on creation
Error: violates PodSecurity "restricted"
```

Explanation

- The **enforce** label applies a standard at admission time.
- **restricted** is the hardened profile (non-root, dropped caps, etc.).
- Non-compliant pods are rejected before they start.

> **Key idea:** Label namespaces with Pod Security Standards to block unsafe pods automatically.

---

084. Pull from a private registry

Scenario — Your images live in a private registry and pods fail with ImagePullBackOff.

Concept — An imagePullSecret holds registry credentials. Referencing it (or attaching it to a ServiceAccount) lets pods authenticate and pull private images.

Command / Code

```
kubectl create secret docker-registry regcred \
  --docker-server=registry.nimbus.io --docker-username=ci --docker-password=$PASS
# pod: spec.imagePullSecrets: [ { name: regcred } ]
```

Output

```
secret/regcred created
# pod now authenticates and pulls the private image
```

Explanation

- **docker-registry** secrets store registry credentials.
- Reference it via **imagePullSecrets** in the pod or ServiceAccount.
- Without it, private pulls fail with ImagePullBackOff.

> **Key idea:** Attach imagePullSecrets to a ServiceAccount so all its pods can pull privately.

Observability & Troubleshooting

085. See live resource usage with kubectl top

Scenario — You need to know which pods are consuming the most CPU and memory right now.

Concept — kubectl top reports live CPU/memory for pods and nodes using the metrics-server. It's the quickest way to spot resource hogs and undersized limits.

Command / Code

```
kubectl top pods --sort-by=memory
kubectl top nodes
```

Output

```
NAME     CPU(cores)   MEMORY(bytes)
db-0     120m         512Mi
web-1    35m          180Mi
worker-1  1200m  45%   6Gi   75%
```

Explanation

- **top pods** shows current usage; **--sort-by** ranks them.
- **top nodes** shows per-node utilisation and percentages.
- It requires the **metrics-server** to be installed.

> **Key idea:** kubectl top needs metrics-server; it's your live usage gauge.

---

086. Read cluster events in order

Scenario — Something went wrong recently and you want a timeline of what happened.

Concept — Events record what controllers and the kubelet did (scheduling, pulling, failing). Sorting them by time gives a chronological account of recent activity.

Command / Code

```
kubectl get events --sort-by=.lastTimestamp | tail -6
```

Output

```
5m   Warning  Failed       pod/web-1   Failed to pull image "nginx:typo"
5m   Normal   BackOff      pod/web-1   Back-off pulling image
2m   Normal   Scheduled    pod/db-0    Successfully assigned...
```

Explanation

- **get events --sort-by** orders events chronologically.
- Warnings usually pinpoint the failure (bad image, failed mount).

- Events are namespaced — add **-A** to see the whole cluster.

> **Key idea:** Sorted events give a timeline — the fastest way to reconstruct an incident.

---

087. Diagnose ImagePullBackOff

Scenario — A pod is stuck in ImagePullBackOff and won't start.

Concept — ImagePullBackOff means the kubelet can't pull the image — a typo, a missing tag, or missing registry credentials. The pod events state the exact cause.

Command / Code

```
kubectl describe pod web-1 | grep -A3 Events
```

Output

```
  Warning  Failed  Failed to pull image "nginx:1.99":
           not found
  Warning  Failed  Error: ImagePullBackOff
```

Explanation

- The event names the image and reason (**not found** here).
- Common causes: wrong tag, private image without credentials, typo.
- Fix the image reference or add an imagePullSecret, then the pod recovers.

> **Key idea:** ImagePullBackOff = image name/tag wrong or credentials missing — read the event.

---

088. Diagnose CrashLoopBackOff

Scenario — A pod keeps restarting with CrashLoopBackOff.

Concept — CrashLoopBackOff means the container starts then exits repeatedly. The application logs (including the previous instance) reveal why it's dying.

Command / Code

```
kubectl get pod app
kubectl logs app --previous | tail -3
```

Output

```
app  0/1  CrashLoopBackOff  5  3m
FATAL: DATABASE_URL is not set
```

Explanation

- **CrashLoopBackOff** is repeated start-then-exit with growing backoff.
- **logs --previous** shows the crashed instance's output.

- Here a missing env var is the root cause — fix config and redeploy.

> **Key idea:** For CrashLoopBackOff, logs --previous almost always reveals the cause.

---

089. Tail logs across many pods

Scenario — A Deployment has several replicas and you want logs from all of them at once.

Concept — kubectl logs with a label selector streams from all matching pods, so you don't chase individual pod names when debugging a multi-replica service.

Command / Code

```
kubectl logs -l app=web --tail=2 --prefix
```

Output

```
[pod/web-1] 10.0.0.5 GET / 200
[pod/web-2] 10.0.0.7 GET /health 200
# --prefix labels each line with its pod
```

Explanation

- **-l app=web** selects all pods of the Deployment.
- **--prefix** tags each line with the source pod.
- Add **-f** to follow all of them live.

> **Key idea:** logs -l --prefix aggregates logs across all replicas.

---

090. Extract fields with jsonpath

Scenario — You need just the image of each pod for a quick audit script.

Concept — Output formats like jsonpath and custom-columns pull specific fields from resources, turning kubectl into a scriptable query tool.

Command / Code

```
kubectl get pods -o jsonpath='{range .items[*]}{.metadata.name}{"\t"}
{.spec.containers[0].image}{"\n"}{end}'
```

Output

```
web-1   nginx:1.27-alpine
web-2   nginx:1.27-alpine
db-0    postgres:16
```

Explanation

- **jsonpath** walks the object tree to extract exact fields.

- The **range ... end** loops over every item in the list.
- **-o custom-columns** is a simpler alternative for tabular output.

> **Key idea:** jsonpath and custom-columns make kubectl scriptable for audits.

---

091. Watch resources change in real time

Scenario — You're rolling out a change and want to watch pods transition live.

Concept — The --watch flag streams updates to a resource list, so you can observe rollouts, scaling, and failures as they happen without re-running the command.

Command / Code

```
kubectl get pods -l app=web --watch
```

Output

```
web-abc  0/1  Pending    0  0s
web-abc  0/1  ContainerCreating  0  2s
web-abc  1/1  Running  0  8s
```

Explanation

- **--watch** keeps the command open, printing each state change.
- You see the pod move Pending → Creating → Running.
- Ctrl-C stops watching; it doesn't affect the cluster.

> **Key idea:** --watch turns any get into a live view of state transitions.

---

092. Discover resources and short names

Scenario — You're unsure of a resource's API group or its short name.

Concept — kubectl api-resources lists every resource type, its API group, whether it's namespaced, and short names — a map of what the cluster can manage.

Command / Code

```
kubectl api-resources | grep -Ei 'deployment|ingress|hpa'
```

Output

```
deployments   deploy   apps/v1        true   Deployment
ingresses     ing      networking...  true   Ingress
horizontalpodautoscalers  hpa  autoscaling  true  HorizontalPodAutoscaler
```

Explanation

- **api-resources** lists types, groups, and short names.

- Short names (deploy, ing, hpa) speed up daily commands.
- The namespaced column tells you if **-n** applies.

> **Key idea:** api-resources reveals short names (deploy, ing, hpa) and API groups.

Advanced: Workloads & Tooling

093. Run stateful pods with a StatefulSet

Scenario — A database cluster needs stable names and per-pod persistent storage.

Concept — A StatefulSet gives pods stable identities (name-0, name-1), ordered rollout, and a unique PVC each via volumeClaimTemplates — unlike interchangeable Deployment pods.

`statefulset.yaml`

```
kind: StatefulSet
metadata: { name: db }
spec:
  serviceName: db
  replicas: 3
  selector: { matchLabels: { app: db } }
  volumeClaimTemplates:
    - metadata: { name: data }
      spec: { accessModes: [ReadWriteOnce], resources: { requests: { storage: 10Gi } } }
```

Output

```
kubectl get pods -l app=db
db-0  1/1  Running
db-1  1/1  Running
db-2  1/1  Running
```

Explanation

- Pods get **stable ordinal names** (db-0, db-1, db-2).
- **volumeClaimTemplates** creates a dedicated PVC per pod.
- A headless **serviceName** gives each pod stable DNS.

> **Key idea:** StatefulSets suit databases: stable identity, per-pod storage, ordered rollout.

---

094. Run one pod per node with a DaemonSet

Scenario — A log collector must run on every node, including new ones.

Concept — A DaemonSet ensures a copy of a pod runs on each (matching) node, automatically adding pods as nodes join. It's used for node-level agents like logging and monitoring.

`daemonset.yaml`

```
kind: DaemonSet
metadata: { name: log-agent }
spec:
  selector: { matchLabels: { app: log-agent } }
  template:
    spec:
      containers: [ { name: agent, image: fluent-bit:2.2 } ]
```

Output

```
kubectl get ds log-agent
NAME        DESIRED  CURRENT  READY  NODE SELECTOR
log-agent   3        3        3      <none>
```

Explanation

- A DaemonSet runs one pod per eligible node.
- When a node joins, it automatically gets the pod.
- Perfect for logging, monitoring, and CNI agents.

> **Key idea:** DaemonSets place a node-level agent on every node, new ones included.

---

095. Run a batch task with a Job

Scenario — You need a one-off database migration that runs to completion, then stops.

Concept — A Job runs pods until a set number complete successfully, retrying failures. Unlike Deployments, Jobs are for finite work, not long-running services.

`job.yaml`

```
kind: Job
metadata: { name: migrate }
spec:
  backoffLimit: 4
  template:
    spec:
      restartPolicy: OnFailure
      containers: [ { name: m, image: nimbus-app:1.0, command: ['./migrate.sh'] } ]
```

Output

```
job.batch/migrate created
kubectl get job migrate
NAME      COMPLETIONS   DURATION
migrate   1/1           22s
```

Explanation

- A **Job** runs its pod until it completes successfully.

- **backoffLimit** caps retries before the Job is marked failed.
- **COMPLETIONS 1/1** shows the task finished.

> **Key idea:** Jobs run finite work to completion; Deployments run services forever.

---

096. Schedule recurring work with a CronJob

Scenario — A nightly backup should run automatically every day at 2 AM.

Concept — A CronJob creates Jobs on a cron schedule, giving you Kubernetes-native periodic tasks like backups, reports, and cleanups.

`cronjob.yaml`

```
kind: CronJob
metadata: { name: backup }
spec:
  schedule: '0 2 * * *'
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: OnFailure
          containers: [ { name: b, image: backup:1.0 } ]
```

Output

```
cronjob.batch/backup created
kubectl get cronjob backup
NAME     SCHEDULE     LAST SCHEDULE
backup   0 2 * * *    8h ago
```

Explanation

- **schedule** uses standard cron syntax.
- Each run spawns a Job from the **jobTemplate**.
- Set history limits and concurrencyPolicy to control overlap and retention.

> **Key idea:** CronJobs create Jobs on a cron schedule for backups, reports, and cleanups.

---

097. Route external HTTP with an Ingress

Scenario — You want host- and path-based routing to several services behind one external IP.

Concept — An Ingress defines HTTP routing rules that an ingress controller implements, giving you name-based virtual hosting and a single entry point instead of many LoadBalancers.

`ingress.yaml`

```
kind: Ingress
metadata: { name: nimbus }
spec:
  ingressClassName: nginx
  rules:
    - host: shop.nimbus.io
      http:
        paths:
          - { path: /, pathType: Prefix, backend: { service: { name: web, port: { number: 80
} } } }
```

Output

```
ingress.networking.k8s.io/nimbus created
kubectl get ingress
NAME     CLASS   HOSTS           ADDRESS
nimbus   nginx   shop.nimbus.io  34.120.10.5
```

Explanation

- **rules** map hosts/paths to backend Services.
- An **ingress controller** (nginx here) actually enforces the routing.
- One Ingress can front many services on a single IP.

> **Key idea:** Ingress needs a controller; it replaces many LoadBalancers with one entry point.

---

098. Install an app with Helm

Scenario — You want to deploy a packaged app without writing every manifest by hand.

Concept — Helm packages Kubernetes manifests into versioned charts with configurable values. Installing a chart creates a named release you can upgrade and roll back as a unit.

Command / Code

```
helm repo add bitnami https://charts.bitnami.com/bitnami
helm install cache bitnami/redis --set architecture=standalone
```

Output

```
"bitnami" has been added to your repositories
NAME: cache
STATUS: deployed
REVISION: 1
```

Explanation

- A **chart** bundles templated manifests plus default values.
- **--set** overrides values without editing files.

- The install creates a tracked **release** named cache.

> **Key idea:** Helm charts package and parameterise manifests into upgradable releases.

---

099. Upgrade and roll back a Helm release

Scenario — A Helm upgrade caused problems and you must revert to the last good state.

Concept — Helm tracks each change as a numbered revision. helm upgrade applies new values or chart versions; helm rollback reverts the release to an earlier revision.

Command / Code

```
helm upgrade cache bitnami/redis --set replica.replicaCount=3
helm history cache
helm rollback cache 1
```

Output

```
Release "cache" has been upgraded. REVISION: 2
REVISION  STATUS
1         superseded
2         deployed
Rollback was a success. Happy Helming!
```

Explanation

- **upgrade** creates a new revision with your changes.
- **history** lists revisions; **rollback N** reverts to one.
- Releases are versioned units, so rollback is clean and atomic.

> **Key idea:** Helm revisions make upgrades and rollbacks a single, tracked operation.

---

100. Manage environments with Kustomize overlays

Scenario — Dev and prod share most manifests but differ in a few values.

Concept — Kustomize (built into kubectl via -k) layers environment-specific overlays on a common base, so you customise without templating or duplicating manifests.

Command / Code

```
# overlays/prod/kustomization.yaml patches replicas + image
kubectl apply -k overlays/prod
kubectl get deploy web -o jsonpath='{.spec.replicas}'
```

Output

```
deployment.apps/web configured
5
# prod overlay set replicas to 5 over the shared base
```

Explanation

- **apply -k** builds and applies a Kustomize overlay.
- The overlay patches a shared **base** with env-specific changes.
- No templating — it's plain YAML plus declarative patches.

> **Key idea:** Kustomize overlays customise a shared base per environment, no templating needed.
